TURN Server Backend API Documentation

Overview

This document provides implementation guide for creating a secure backend API to generate TURN server credentials. This approach keeps the shared secret secure on the server side instead of exposing it in the mobile app.

Why Backend API?

❌ Current Approach (Client-Side)

```
// Secret exposed in .env file
TURN_SHARED_SECRET=static-auth-secret  // ⚠️ Can be extracted from APK
```

Problems:

- Secret can be extracted by decompiling APK
- No control over credential generation
- Cannot revoke access
- No user tracking

☑ Recommended Approach (Backend API)

```
// Secret stays on server
const secret = process.env.TURN_SHARED_SECRET;  // ☑ Secure
```

Benefits:

- ☑ Secret remains secure on server
- ☑ Can revoke/change secret anytime
- ☑ Track which users are using TURN
- ☑ Rate limiting and abuse prevention
- ☑ Dynamic server allocation

Backend Implementation

Option 1: Node.js (Express)

```
const express = require('express');
const crypto = require('crypto');
const app = express();
```

```
// Middleware for authentication
const authenticate = (req, res, next) => {
  const token = req.headers.authorization?.replace('Bearer ', '');

  if (!token) {
    return res.status(401).json({ error: 'Unauthorized' });
  }

  // Verify JWT token and extract user info
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (error) {
    return res.status(401).json({ error: 'Invalid token' });
  }
};

// TURN credentials endpoint
app.post('/api/turn-credentials', authenticate, (req, res) => {
  const userId = req.user.id;
  const ttl = 86400; // 24 hours in seconds
  const expiryTime = Math.floor(Date.now() / 1000) + ttl;
  const username = `${expiryTime}:${userId}`;

  // Server-side secret (safe)
  const secret = process.env.TURN_SHARED_SECRET;

  // Generate HMAC-SHA1 password
  const password = crypto
    .createHmac('sha1', secret)
    .update(username)
    .digest('base64');

  // Log for monitoring (optional)
  console.log(`TURN credentials generated for user: ${userId}`);

  res.json({
    username,
    password,
    ttl,
    urls: [
      'stun:stun.l.google.com:19302',
      'stun:stun1.l.google.com:19302',
      'turn:call.be-mindpower.net?transport=udp',
      'turn:call.be-mindpower.net?transport=tcp'
    ]
  });
});

app.listen(3000, () => {
  console.log('TURN API running on port 3000');
});
```

Option 2: Python (Flask)

```python
from flask import Flask, request, jsonify
import hmac
import hashlib
import base64
import time
import os
from functools import wraps

app = Flask(__name__)

def authenticate(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        token = request.headers.get('Authorization', '').replace('Bearer ', '')

        if not token:
            return jsonify({'error': 'Unauthorized'}), 401

        # Verify token and extract user info
        try:
            # Add your JWT verification logic here
            user_id = verify_jwt_token(token)
            request.user_id = user_id
            return f(*args, **kwargs)
        except Exception as e:
            return jsonify({'error': 'Invalid token'}), 401

    return decorated_function

@app.route('/api/turn-credentials', methods=['POST'])
@authenticate
def get_turn_credentials():
    user_id = request.user_id
    ttl = 86400  # 24 hours
    expiry_time = int(time.time()) + ttl
    username = f"{expiry_time}:{user_id}"

    # Server-side secret (safe)
    secret = os.getenv('TURN_SHARED_SECRET').encode('utf-8')

    # Generate HMAC-SHA1 password
    password = base64.b64encode(
        hmac.new(secret, username.encode('utf-8'), hashlib.sha1).digest()
    ).decode('utf-8')

    # Log for monitoring (optional)
    print(f"TURN credentials generated for user: {user_id}")
```

```
    return jsonify({
        'username': username,
        'password': password,
        'ttl': ttl,
        'urls': [
            'stun:stun.l.google.com:19302',
            'stun:stun1.l.google.com:19302',
            'turn:call.be-mindpower.net?transport=udp',
            'turn:call.be-mindpower.net?transport=tcp'
        ]
    })

if __name__ == '__main__':
    app.run(port=3000)
```

Option 3: PHP (Laravel)

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class TurnCredentialsController extends Controller
{
    public function __construct()
    {
        $this->middleware('auth:api');
    }

    public function getCredentials(Request $request)
    {
        $userId = Auth::id();
        $ttl = 86400; // 24 hours
        $expiryTime = time() + $ttl;
        $username = "{$expiryTime}:{$userId}";

        // Server-side secret (safe)
        $secret = env('TURN_SHARED_SECRET');

        // Generate HMAC-SHA1 password
        $password = base64_encode(
            hash_hmac('sha1', $username, $secret, true)
        );

        // Log for monitoring (optional)
        \Log::info("TURN credentials generated for user: {$userId}");

        return response()->json([
            'username' => $username,
```

```
            'password' => $password,
            'ttl' => $ttl,
            'urls' => [
                'stun:stun.l.google.com:19302',
                'stun:stun1.l.google.com:19302',
                'turn:call.be-mindpower.net?transport=udp',
                'turn:call.be-mindpower.net?transport=tcp'
            ]
        ]);
    }
}
```

Flutter App Implementation

1. Create Service Class

```dart
// lib/services/turn_credentials_service.dart

import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:flutter_dotenv/flutter_dotenv.dart';

class TurnCredentialsService {
  static String get baseUrl => dotenv.env['CUSTOMER_API_BASE_URL'] ?? '';

  /// Fetch TURN credentials from backend
  static Future<Map<String, dynamic>> fetchCredentials(String token) async {
    try {
      final response = await http.post(
        Uri.parse('$baseUrl/turn-credentials'),
        headers: {
          'Authorization': 'Bearer $token',
          'Content-Type': 'application/json',
        },
      );

      if (response.statusCode == 200) {
        return json.decode(response.body);
      } else {
        print('Failed to fetch TURN credentials: ${response.statusCode}');
        return {};
      }
    } catch (e) {
      print('Error fetching TURN credentials: $e');
      return {};
    }
  }
}
```

2. Update TurnCredentials Class

```
// lib/shared/utils/voip/turn_credentials.dart

import 'dart:async';
import 'package:flutter_webrtc/flutter_webrtc.dart' as webrtc;
import '../../../services/turn_credentials_service.dart';
import 'voip_logger.dart';

class TurnCredentials {
  /// Get ICE servers configuration from backend
  static Future<List<Map<String, dynamic>>> getIceServers(String token) async {
    final credentials = await TurnCredentialsService.fetchCredentials(token);

    if (credentials.isEmpty) {
      VoipLogger().logWarning('Failed to fetch TURN credentials from backend');
      return _getFallbackServers();
    }

    VoipLogger().logInfo('✅ TURN credentials fetched from backend');
    VoipLogger().logInfo('   Username: ${credentials['username']}');
    VoipLogger().logInfo('   TTL: ${credentials['ttl']} seconds');

    final urls = credentials['urls'] as List<dynamic>;
    final stunUrls = urls.where((url) =>
url.toString().startsWith('stun:')).toList();
    final turnUrls = urls.where((url) =>
url.toString().startsWith('turn:')).toList();

    return [
      if (stunUrls.isNotEmpty)
        {
          'urls': stunUrls,
        },
      if (turnUrls.isNotEmpty)
        ...turnUrls.map((url) => {
          'urls': [url],
          'username': credentials['username'],
          'credential': credentials['password'],
        }),
    ];
  }

  /// Fallback to Google STUN only if backend fails
  static List<Map<String, dynamic>> _getFallbackServers() {
    VoipLogger().logWarning('Using fallback STUN servers (no TURN)');
    return [
      {
        'urls': [
          'stun:stun.l.google.com:19302',
          'stun:stun1.l.google.com:19302',
        ],
```

```
      },
    ];
  }

  /// Test connectivity with backend credentials
  static Future<bool> testConnectivity(String token) async {
    VoipLogger().logInfo('');

VoipLogger().logInfo('╔═══════════════════════════════════════════════════════════╗
');
    VoipLogger().logInfo('║         TURN SERVER CONNECTIVITY TEST
║');

VoipLogger().logInfo('╚═══════════════════════════════════════════════════════════╝
');

    try {
      final iceServers = await getIceServers(token);

      if (iceServers.isEmpty) {
        VoipLogger().logError('❌ No ICE servers available');
        return false;
      }

      VoipLogger().logInfo('🌐 Creating WebRTC PeerConnection...');
      VoipLogger().logInfo('   └─ ICE Servers count: ${iceServers.length}');

      final config = {
        'iceServers': iceServers,
        'iceCandidatePoolSize': 0,
      };

      final pc = await webrtc.createPeerConnection(config);

      final completer = Completer<bool>();
      bool hasRelay = false;
      bool hasStun = false;
      bool hasHost = false;

      VoipLogger().logInfo('🧊 Gathering ICE Candidates...');

      pc.onIceCandidate = (candidate) {
        if (candidate.candidate != null && candidate.candidate!.isNotEmpty) {
          final candidateStr = candidate.candidate!;

          if (candidateStr.contains('typ relay')) {
            hasRelay = true;
            VoipLogger().logInfo('   └─ Found: relay (TURN) ✅');
          } else if (candidateStr.contains('typ srflx')) {
            hasStun = true;
            VoipLogger().logInfo('   └─ Found: srflx (STUN) ✅');
          } else if (candidateStr.contains('typ host')) {
            hasHost = true;
            VoipLogger().logInfo('   └─ Found: host (local) ✅');
```

```
            }
          }
        };

        pc.onIceGatheringState = (state) {
          if (state == webrtc.RTCIceGatheringState.RTCIceGatheringStateComplete) {
            if (!completer.isCompleted) {
              completer.complete(hasRelay);
            }
          }
        };

        await pc.createDataChannel('test', webrtc.RTCDataChannelInit());
        final offer = await pc.createOffer();
        await pc.setLocalDescription(offer);

        final result = await completer.future.timeout(
          const Duration(seconds: 10),
          onTimeout: () => hasRelay,
        );

        await pc.close();

        VoipLogger().logInfo('');

  VoipLogger().logInfo('╔═══════════════════════════════════════════════════════════╗
');
        VoipLogger().logInfo('│              TEST RESULTS
│');

  VoipLogger().logInfo('╠═══════════════════════════════════════════════════════════╣
');
        VoipLogger().logInfo('│ Host (Local):  ${hasHost ? "✅" : "❌"}');
        VoipLogger().logInfo('│ STUN (Public): ${hasStun ? "✅" : "❌"}');
        VoipLogger().logInfo('│ TURN (Relay):  ${hasRelay ? "✅" : "❌"}');

  VoipLogger().logInfo('╚═══════════════════════════════════════════════════════════╝
');

        return result;
      } catch (e, s) {
        VoipLogger().logError('❌ Test failed', e, s);
        return false;
      }
    }
  }
```

3. Usage in VoIP Call

```
// When initiating a call
final token = await getAuthToken(); // Your auth token
```

```
final iceServers = await TurnCredentials.getIceServers(token);

final config = {
  'iceServers': iceServers,
  'sdpSemantics': 'unified-plan',
};

final peerConnection = await createPeerConnection(config);
```

API Response Format

Success Response (200 OK)

```
{
  "username": "1734278400:user123",
  "password": "dGVzdHBhc3N3b3Jk",
  "ttl": 86400,
  "urls": [
    "stun:stun.l.google.com:19302",
    "stun:stun1.l.google.com:19302",
    "turn:call.be-mindpower.net?transport=udp",
    "turn:call.be-mindpower.net?transport=tcp"
  ]
}
```

Error Response (401 Unauthorized)

```
{
  "error": "Unauthorized"
}
```

Error Response (500 Internal Server Error)

```
{
  "error": "Failed to generate credentials"
}
```

Environment Variables

Backend (.env)

```
# TURN Server Configuration
TURN_SHARED_SECRET=your_super_secret_key_here
TURN_SERVER_URL=call.be-mindpower.net

# JWT Secret for authentication
JWT_SECRET=your_jwt_secret_here
```

Flutter App (.env)

```
# Remove TURN_SHARED_SECRET from here!
# Only keep API base URL
CUSTOMER_API_BASE_URL=https://dev-backend.be-mindpower.net/api
```

Security Best Practices

1. **Always use HTTPS** for API communication
2. **Validate JWT tokens** on every request
3. **Rate limit** the endpoint (e.g., 10 requests per minute per user)
4. **Log credential generation** for monitoring
5. **Rotate shared secret** periodically
6. **Set appropriate TTL** (not too long, not too short)
7. **Monitor usage** to detect abuse

Testing

Test Backend API

```
# Using curl
curl -X POST https://your-api.com/api/turn-credentials \
  -H "Authorization: Bearer YOUR_JWT_TOKEN" \
  -H "Content-Type: application/json"
```

Test in Flutter App

```
// Run connectivity test
final token = await getAuthToken();
final success = await TurnCredentials.testConnectivity(token);
print('TURN test result: $success');
```

Migration Steps

1. ☑ Implement backend API endpoint
2. ☑ Deploy backend with TURN_SHARED_SECRET
3. ☑ Update Flutter app to use new service
4. ☑ Test in development environment
5. ☑ Remove TURN_SHARED_SECRET from app .env
6. ☑ Deploy app update
7. ☑ Monitor logs for issues

Troubleshooting

Issue: "Failed to fetch TURN credentials"

- Check if backend API is running
- Verify JWT token is valid
- Check network connectivity

Issue: "TURN server unreachable"

- Verify TURN server is running (coturn)
- Check firewall rules (ports 3478, 5349)
- Verify shared secret matches on server

Issue: "Only STUN working, no TURN"

- Check TURN server logs
- Verify credentials are correct
- Test with `turnutils_uclient` tool

Additional Resources

- Coturn Server Setup
- WebRTC ICE Documentation
- TURN Server Testing Tools

Support

For issues or questions, contact the backend team or refer to the main project documentation.